

# Kapitel 21. Överlagring av operatorer

## Innehållsförteckning

[Vad är överlagring av operatorer?](#)

[Överlagring av operatörn +](#)

[Överlagring av operatörn -](#)

[Överlagring och friends](#)

[Operatorerna == och =](#)

[Överlagring av << och >>](#)

[Överlagring av typkonverteringar](#)

[Diskussion](#)

Detta kapitel behandlar hur olika operatorer kan överlagras för att göra klasser mera transparenta för programmeraren.

## Vad är överlagring av operatorer?

I ett tidigare kapitel behandlade vi hur funktioner och metoder kan överlagras med hjälp av *polymorfism* (se [avsnittet Polymorfism i Kapitel 17](#)). Överlagring av metoder låter oss ha metoder med samma namn men som tar olika parametrar. På samma sätt kan man även överlagra *operatorer* i C++ och lägga till eller ändra funktionalitet. Vi kan på detta sätt definiera olika operatorer som annars inte kunde användas med någon viss klass att ha en meningsfull betydelse. Genom detta kan man göra olika klasser mera *transparenta* för användaren. De blir lättare och mera intuitiva att använda och påminner mera om primitiva datatyper såsom t.ex. `int` eller `double`.

## Operatorer som kan överlagras

En operator i C++ är en "symbol" som opererar på en datatyp. Det mesta av den text som binder ihop datatyper är olika operatorer. De olika operatorer som kan överlagras i C++ är följande:

**Tabell 21-1. Operatorer som kan överlagras**

|    |    |     |     |       |        |          |
|----|----|-----|-----|-------|--------|----------|
| +  | -  | *   | /   | %     | ^      | &        |
|    | ~  | !   | =   | <     | >      | +=       |
| -= | *= | /=  | %=  | ^=    | &=     | =        |
| << | >> | <<= | >>= | ==    | !=     | <=       |
| >= | && |     | ++  | □     | ->*    | ,        |
| -> | [] | ()  | new | new[] | delete | delete[] |

Inga andra operatorer än de ovannämnda kan överlagras, men de flesta som är användbara finns bland dessa. Man vill relativt sällan överlagra andra än normala räkneoperatorer, jämförelse- och tilldelningsoperatorerna. Avancerade klasser som vill ha egen minneshantering kan dock överlagra t.ex. `new` och `delete`, men det ämnet går utanför detta kompendiums omfång. Se kursboken för mera information.

Då operatorer överlagras måste de bibehålla samma antal parametrar som de normalt gör. Man kan sålunda inte definiera en divisionsoperator (`/`) som tar en eller tre operander (parametrar).

Divisionsoperatorn skall ha två operander. Däremot kan t.ex. operatorn minus (-) mycket väl ha en eller två parametrar, d.v.s. normal subtraktion (två parametrar) och negation (en parameter). Man kan inte ändra operatorers precedensordning från vad den är i normala fall. Enda möjligheten är att använda parenteser.

Det är inte heller vettigt att ändra på en operators funktionalitet genom överlagring. Det är helt möjligt att överlagra tilldelningsoperatorn till att skriva ut ett objekt på skärmen, men det är inte vettigt. Användaren har en grunduppfattning om vad som borde hända då t.ex. = används, och man bör inte ändra betydelsen. Man skall bibehålla operatorers normala betydelse, och endast göra funktionaliteten bättre om den redan finns, och implementera den normala funktionaliteten om operatorn inte redan kan användas tillsammans med klassen i fråga.

## Exempelklassen `Vector`

I detta kapitel skall vi behandla en klass `Vector` som ett genomgående exempel. Vi kommer att börja från en enkel klass och därefter överlagra olika operatorer för att göra klassen användbar och transparent. Klassen `Vector` representerar en matematisk vektor i rymdgeometri. Den innehåller x-, y- och z-koordinater samt metoder för att accessera dessa. En vektor definierar således en viss riktning och har en viss längd. Alla vektorer antas starta från origo, d.v.s. punkten (0, 0, 0).

Vi får en första definition av klassen `Vector` enligt följande:

```
#ifndef VECTOR_H
#define VECTOR_H

#include <iostream>
#include <math.h>

class Vector {
public:
    // konstruktörer
    Vector ();
    Vector (const Vector & V);
    Vector (const float X, const float Y, const float Z);

    // accessera individuella komponenter
    void setX (const float X) { m_X = X; };
    void setY (const float Y) { m_Y = Y; };
    void setZ (const float Z) { m_Z = Z; };
    float x () const { return m_X; };
    float y () const { return m_Y; };
    float z () const { return m_Z; };

    // get the length of the vector
    float length () const;

private:
    // x, y och z-komponenterna
    float m_X;
    float m_Y;
    float m_Z;
};

#endif // VECTOR_H
```

Koden ovan finns i en headerfil `Vector1.h`. Implementationen av metoderna finns i filen `Vector1.cpp`. Den ser i grundutförande ut på följande sätt:

```
#include "Vector1.h"

Vector::Vector () {
    // nollställ alla medlemmar
    m_X = 0;
    m_Y = 0;
    m_Z = 0;
}

// copy-konstruktor
Vector::Vector (const Vector & V) {
    // kopiera data från 'V'
    m_X = V.m_X;
    m_Y = V.m_Y;
    m_Z = V.m_Z;
}

// skapa från separata värden
Vector::Vector (const float X, const float Y, const float Z) {
    // spara värden i medlemmar
    m_X = X;
    m_Y = Y;
    m_Z = Z;
}

// beräkna längden av vektorn
float Vector::length () const {
    // använd Pythagoras
    return sqrt ( m_X * m_X + m_Y * m_Y + m_Z * m_Z );
}
```

Senare då vi fyller på nya metoder skrivs inte båda filerna ut i sin helhet, utan endast prototypen på den nya (eller ändrade) metoden, samt implementationen om en dylik finns. Dessa skall då placeras i header- respektive implementationsfilen på korrekt plats. Korrekt plats i headerfilen är i klassens `public:-`avsnitt. Den kompletta klassen när den är färdig finns i [Appendix C](#).

---

[Förutgående](#)

Hierarkier med exceptions

[Hem](#)[Nästa](#)

Överlagring av operatör +